



BSc (Hons) in **Information Technology**  
**Specialized in AI**  
**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 03

**Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

### Part 1: Practical Implementation — Building the Search Agent

#### Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week\_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:  

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

#### Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS:** Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()` ). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()` ). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()` ) ordered by the total path cost  $g(n)$ .
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

### Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

## Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. **(Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?**

The state space is the set of all possible states connected by actions. Here that is every open (x, y) cell on the grid, with moves as edges. It is a fixed graph in which each state appears only once. The search tree is what the algorithm builds while searching: its root is the start state and each node represents a path to a state. The same state can appear in many nodes (reached by different paths), so the tree can be much larger than the state space, and even infinite if paths loop.

**2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?**

It solves the problem of repeated states: redundant paths and loopy paths (cycles). A state that has already been reached is not added to the frontier again. This turns tree search into graph search. Without it, my DFS agent could step Up, then Down, then Up again forever between two cells (it always expands the newest node). It would never finish, its stack would keep growing, and it could get stuck in an infinite loop without ever finding food

**3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?**

BFS uses a FIFO queue, so it expands nodes level by level: every path of length 1, then 2, and so on. The first time it reaches the food is therefore via the fewest steps. Every step in this grid costs 1, so fewest steps = lowest cost, which makes BFS optimal here (the lecture: optimal if cost = 1 per step).

DFS uses a LIFO stack and always dives down the newest branch as far as it can, then returns the first path it finds to the goal, not the shortest one. In the simulation this showed as long, snaking routes: DFS needed about 290 moves and still left 4-6 food, while BFS cleared all 15 food in 47-60 moves. UCS matched BFS because all step costs are equal.

**4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.**

BFS: time  $O(bd+1)$  and space  $O(b^{(d+1)})$ , because it keeps every generated node in memory. The whole frontier (the full "ring" of cells at the current depth) plus every reached cell is stored at once. On a 1,000,000-cell grid that can mean up to a million stored nodes, and my version stores a full action list with each node, so memory explodes before a far-away food is found. UCS has the same problem:  $O(b^{(1+C^*/\epsilon)})$  time and space.

DFS: time  $O(bm)$  but space only  $O(bm)$ , linear in depth, because it only needs the current path plus the unexpanded siblings along it. So DFS is the memory-friendly option. The trade-off is that  $m$  can be enormous on a 1000x1000 grid (a path can snake through almost every cell), and the solution it returns is not optimal.

Conclusion: the BFS/UCS bottleneck is breadth: the frontier grows exponentially with depth. The DFS bottleneck is depth: memory stays small, but the paths are long and wasteful.

**Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project**

Finalize and Lock Lab 03 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**



**FACULTY OF COMPUTING**